Audit Report

Own Protocol v2

Date: **July 5 – July 12, 2026**

Introduction

Own is a permissionless protocol for bringing tokenized real-world assets (RWAs) onchain. Users mint ERC-20 tokens called **eTokens** (e.g. eTSLA, eGOLD) using stablecoins; each eToken tracks the price of its underlying asset through onchain oracles and is backed by on-chain collateral held in Own Vaults. Where traditional RWA tokenization concentrates trust in a single custodian holding the physical asset, Own backs exposure with transparent, code-governed collateral vaults: any hedge fund, trading firm, or RWA custodian can permissionlessly operate a vault and back issued eTokens, with competition between them driving spreads.

This second review covers the PSM (Peg Stability Module) and RWA reserve-vault extensions built on the `psm-module` branch, on top of the previously audited core (see Audit Report 1, commit `9103c59`). These extensions complete the protocol's layered backing design and address its capital efficiency: in the core system, issuing $1 of eToken exposure requires roughly 3× capital — about 2× LP collateral behind the global utilization cap plus 1× maker capital for the off-chain hedge. With the PSM stack, the protocol instead holds each listed asset's wrapper token itself (e.g. a regulated issuer's tokenized TSLA behind eTSLA) in a protocol-owned, share-less `ReserveVault`. Wrapper collateral is delta-one with the eToken exposure it backs — its price moves with the liability — so it backs issuance 1:1 with no overcollateralization, and a fully reserved eToken consumes no LP capital. The crypto-collateral vaults shrink to a buffer covering only the residual: exposure minted via RFQ that a maker has not yet backfilled with wrapper tokens. Steady-state capital is ≈ 1× productive wrapper reserve plus a small crypto buffer sized to in-flight volume, and the reserve is never LP equity — every unit of reserve is matched by a unit of eToken liability.

The new surface under review comprises: `OwnMarket.psmMint` / `psmRedeem` — permissionless two-way 1:1 conversion between wrapper tokens and eTokens at a derived conversion ratio with a fail-closed ratio-jump guard; `OwnMarket.psmFillOrder` — permissionless atomic delivery-vs-payment fills of resting orders against the PSM reserve, with a per-asset operator kill switch; `ReserveVault` — wrapper custody, maker backfill and recovery, and manager-bound skim/sweep; vault classes with per-asset delta netting in `VaultManager`; four per-asset `AssetRegistry` allowlists (all default-deny, armed only by the deploy scripts); and the Own lending pool (`OwnLendingPool`, `LendingRouter`, `OwnAToken`/`OwnDebtToken`) with `VaultYieldManager`.

The review combined manual analysis of the new PSM and lending contracts, a dedicated review of the reserve release path, a full multi-agent automated audit pass over the entire `src/` tree, and regression review of the new code's interactions with the audited core — including the revised force-execute vault design (an admin-approved pool with redeemer choice, replacing the single designated vault; the underlying escrow bound and health-factor gates from Report 1 are unchanged). Fixes carry mutation-checked regression tests, and the PSM invariant campaign (resting orders, partial DvP fills, cancels, fill pause, escrow conservation) runs 8 invariants green at 1000 runs × depth 100. Full suite: 989 tests passing.

Scope

Repository: `own-protocol/own-v2`

Audited Commit: `d1945aa6acdc0d416ed646e2ef8aca4e92a6914e`

Files:

- `src/core/AssetRegistry.sol`
- `src/core/BorrowManager.sol`
- `src/core/OracleVerifier.sol`
- `src/core/OwnLendingPool.sol`
- `src/core/OwnMarket.sol`
- `src/core/OwnVault.sol`
- `src/core/ProtocolRegistry.sol`
- `src/core/PythOracleVerifier.sol`
- `src/core/ReserveVault.sol`
- `src/core/VaultManager.sol`
- `src/libraries/InterestRateModel.sol`
- `src/libraries/LendingMath.sol`
- `src/periphery/LendingRouter.sol`
- `src/periphery/VaultYieldManager.sol`
- `src/periphery/WETHRouter.sol`
- `src/periphery/WstETHRouter.sol`
- `src/tokens/EToken.sol`
- `src/tokens/ETokenFactory.sol`
- `src/tokens/OwnAToken.sol`
- `src/tokens/OwnDebtToken.sol`

Findings

Each issue has an assigned severity:

- **Critical/High** issues are directly exploitable security vulnerabilities that can lead to loss of funds and need to be fixed.

- **Medium** issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- **Low/Info** issues are non-exploitable or informational findings that do not pose a direct security risk to the system's integrity.

Issues marked RESOLVED are fixed in the codebase and carry a regression test that fails without the fix. Issues marked ACKNOWLEDGED are accepted by design, with the rationale and any operational mitigations documented.

Issues Found

Critical	High	Medium	Low/Info
0	0	2	4

Issues Not Fixed and Not Acknowledged

Critical	High	Medium	Low/Info
0	0	0	0

Summary of Findings

ID	Severity	Title	Status
M-01	Medium	`ReserveVault` surplus guard compared a fresh collateral mark against a stale exposure mark	Fixed
M-02	Medium	`OwnVault.sweepToken` could sweep the vault's own share-token escrow, draining the withdrawal queue	Fixed
I-01	Info	`redeemHalted` can round a dust redemption to a zero payout	Acknowledged

I-02	**Info**	`applySplit` sub-dust exposure drift across migrations	Acknowledged
I-03	**Info**	ETH sent directly to the oracle verifier above the exact fee is trapped	Acknowledged
I-04	**Info**	Protocol-wide `OPERATOR` role scope vs per-contract documentation	Acknowledged

Issue M-01: `ReserveVault` surplus guard compared a fresh collateral mark against a stale exposure mark RESOLVED

Severity: **Medium**

Vulnerability Detail

`ReserveVault._releaseCollateral` (backing `withdraw` — maker recovery — and `skimExcess`) re-marked only the collateral leg before its surplus guard: `pullCollateralPrice` valued the reserve at the current wrapper oracle price × live balance, but the exposure leg read `_assetExposureUSD`, which is only rewritten by exposure changes, keeper price pulls, or a halt — never on this path. No freshness gate applied, and the wrapper oracle's stored price updates independently of the keeper cadence, so the two legs diverged without bound during keeper lag.

Trace: 1,000 eTSLA exposure at mark $100 (exposure $100k), reserve 1,000 ondoTSLA; TSLA rallies to $110 and the wrapper oracle posts $110 with no keeper pull yet. An allowlisted maker's `withdraw` re-marks collateral to $110k against the stale $100k exposure — the guard permits releasing ~$10k of wrapper. The next keeper pull re-marks exposure to $110k against a $100k reserve: the 1:1 backing is short $10k, and the shortfall folds silently into the generic collateral pool.

Impact

An allowlisted maker or operator could — even innocently — skim reserve that was not surplus during keeper lag, under-backing eToken holders; funds exit to the maker's settlement address and are not governance-recoverable.

Resolution

`_releaseCollateral` calls `vmgr.pullAssetPrice(backed)` (permissionless; halt-aware) before the guard, so both legs read current stored oracle prices in the same transaction. Residual divergence is bounded by oracle posting cadence rather than keeper cadence — the same trust level as the PSM conversion ratio. Fail-closed side effect: a release reverts `PriceUnavailable` while the backed asset's oracle price is unset. Both regression tests are mutation-checked against the pre-fix code.

Issue M-02: `OwnVault.sweepToken` could sweep the vault's own share-token escrow, draining the withdrawal queue

RESOLVED

Severity: **Medium**

Vulnerability Detail

`OwnVault.sweepToken` (stray-token recovery, manager/operator-gated, added on this branch) guarded only `token == asset()`, then swept the vault's *entire* balance of `token` to the caller. But an ERC-4626 vault is its own share token, and `requestWithdrawal` escrows shares via a self-transfer, so the vault's balance of its own shares equals the pending-withdrawal escrow. Calling `sweepToken(address(this))` therefore transferred the whole withdrawal-queue escrow out: afterwards `fulfillWithdrawal` and `cancelWithdrawal` revert on the missing balance, permanently bricking every pending withdrawal, and the swept shares are fungible LP claims redeemable for collateral. No malice required — an honest operator recovering genuinely-stray shares would take the escrow with them.

Impact

Every pending withdrawal irreversibly bricked and the escrowed LP claims redeemable by the caller; privileged (non-timelocked OPERATOR) trigger, loss not governance-recoverable.

Resolution

`sweepToken` reserves the tracked escrow before sweeping the share token (`if (token == address(this)) amount -= _pendingWithdrawalShares;`). Only the surplus above the tracked escrow — a genuine stray transfer — is recoverable; an escrow-only balance reverts `ZeroAmount`. `_pendingWithdrawalShares` moves in lockstep with the escrow, so the subtraction cannot underflow. Regression tests are mutation-checked; stray-share recovery (the function's purpose) is preserved.

Low / Informational Findings

I-01: `redeemHalted` can round a dust redemption to a zero payout

ACKNOWLEDGED

A sub-dust `redeemHalted` amount can floor to a zero payout while the eTokens are still burned. Accepted — self-inflicted, bounded to dust; a minimum-amount gate on a wind-down path would add a revert surface where liveness matters most.

I-02: `applySplit` sub-dust exposure drift across migrations ACKNOWLEDGED

Repeated split re-denominations can accumulate sub-dust drift in recorded exposure. Observed drift is protocol-favorable and bounded; accepted, with a dedicated multi-split fuzz pass tracked as future work.

I-03: ETH sent directly to the oracle verifier above the exact fee is trapped

ACKNOWLEDGED

Both protocol consumers forward the exact verification fee and refund the remainder to the caller; only a third party calling `verifyPrice` directly with excess ETH can strand it — their own overpayment. Accepted.

I-04: Protocol-wide `OPERATOR` role scope vs per-contract documentation

ACKNOWLEDGED

The `OPERATOR` role is granted protocol-wide while some per-contract NatSpec read as if scoped per contract; likewise the `haltAsset` price and redemption ratio are operator-trust levers by design. Documentation aligned; no code change.